

美团 AI 面试专项准备

使用方式

这份材料按“先能讲出口，再能扛追问”来准备。面试时不要背长段，优先用三段式回答：

1. 先给结论。
2. 再结合项目或工程场景。
3. 最后主动说边界和可扩展方案。

项目相关回答必须遵守 00_CONTEXT_LOCK.md：当前项目没有实现 WebSocket、生产者 confirm、死信队列、本地消息表、完整秒杀补偿闭环和多机分布式限流，面试中只能作为扩展方案讲。

1. 自我介绍

60 秒版本

面试官您好，我叫张雨豪，来自武汉大学。我的主要技术方向是后端开发，同时也做过 Vue 前端配合，比较熟悉 Go、Gin、GORM、MySQL、Redis、RabbitMQ 这些技术栈。

我最近重点做的是一个校园电商秒杀系统。这个项目不是只做普通 CRUD，而是围绕高并发下单链路做了 Redis 库存预热、Lua 原子预扣、RabbitMQ 异步削峰、consumer 幂等落库、失败回滚、基础库存补偿、限流和 Prometheus 监控。我还写了 Node.js 压测脚本，观察 RabbitMQ ready backlog，并通过调整 consumer worker 和 prefetch 参数提升消费吞吐。

我平时也会把 AI 工具融入学习和开发，主要用于拆解技术方案、定位 bug、生成测试思路和做代码 review，但核心代码和设计决策会自己理解后再落地。我希望实习中能继续做偏工程落地和系统稳定性的工作。

如果面试官追问“你最熟悉哪条链路”

我最熟悉下单和秒杀链路。请求进来后先在 Redis 用 Lua 做库存、token、限购的原子校验和预扣，成功后生成 Snowflake 订单 ID，再投递 RabbitMQ。consumer 端用手动 ack，事务里完成库存扣减、余额扣减、订单和订单项创建、销量更新。为了防止重复消费，consumer 会先按订单 ID 做幂等检查，DB 层也用主键兜底。这个链路的核心是“Redis 前置拦截、MQ 削峰、DB 条件扣减兜底、失败补偿收敛不一致”。

2. AI 如何融入学习和实践

可直接回答

我使用 AI 不是让它直接替我写完整代码，而是把它当成“结对工程助手”。主要有四类场景：

- 学习新技术时，让 AI 帮我建立知识地图，比如 Redis Lua、RabbitMQ ack、prefetch、MVCC 这些概念之间的关系。
- 做方案设计时，让 AI 帮我列异常分支，例如 Redis 预扣成功但 MQ 投递失败、MQ 成功但 DB 失败、consumer 重复消费这些情况。
- 编码时，让 AI 生成局部代码思路或测试用例，然后我结合项目结构自己改。
- 调优和复盘时，让 AI 帮我设计压测指标，比如 QPS、P95/P99、RabbitMQ ready/unacked、DB 连接池、Redis 延迟等。

我会特别注意验证 AI 输出，不会直接相信。比如涉及项目能力边界时，我会回到代码确认有没有真的实现，避

免把扩展方案说成已实现。

AI 发挥特别大作用的项目经历

在校园电商秒杀系统里，AI 对我帮助最大的是“把高并发下单从同步 DB 写改造成 Redis + MQ 异步链路”的设计拆解。

一开始我只知道秒杀要用 Redis 扣库存，但不清楚异常链路怎么闭环。我让 AI 帮我枚举风险点，发现关键问题不只是扣库存，还包括 Lua 成功但 MQ 投递失败、consumer 消费失败、MQ 重复投递、Redis 和 MySQL 短暂不一致、压测时消息积压等。之后我把方案拆成几个模块逐步实现：Redis Lua 原子预扣、投递失败回滚、RabbitMQ durable queue 和 persistent message、consumer 手动 ack、订单 ID 幂等、库存补偿和压测脚本。

AI 的作用主要是帮我更系统地暴露问题和组织学习路径，但每个实现我都会回到 Go 代码、日志和压测结果里验证。

3. HTTP 与 HTTPS

HTTP 和 HTTPS 的区别

HTTP 是明文传输，默认 80 端口，不提供加密、身份认证和完整性保护。HTTPS 是 HTTP over TLS，默认 443 端口，在 HTTP 和 TCP 之间加入 TLS 层，提供三件事：加密传输、防篡改、服务器身份认证。

更具体地说，HTTPS 不是替换 HTTP 语义，HTTP 的方法、状态码、Header 仍然存在，只是传输前被 TLS 加密保护。

HTTPS 如何保证安全连接

HTTPS 主要靠四类技术：

- 非对称加密：用于身份认证和密钥协商，例如 RSA、ECDHE。
- 对称加密：用于正式业务数据传输，例如 AES-GCM、ChaCha20-Poly1305。
- 数字证书：由 CA 签发，证明公钥属于目标域名。
- 完整性校验：通过 MAC 或 AEAD 机制防止数据被篡改。

非对称加密适合建立信任和交换密钥，但性能低；对称加密性能高，所以真正传业务数据时使用协商出来的会话密钥。

TLS 握手机制

以 TLS 1.2/1.3 的通用思路来说：

1. ClientHello: 客户端发送支持的 TLS 版本、随机数、加密套件、SNI 等。
2. ServerHello: 服务端选择协议版本和加密套件，返回服务端随机数和证书。
3. 证书校验: 客户端验证证书链、域名、有效期、签名。
4. 密钥协商: 双方通过 RSA 或更常见的 ECDHE 协商出共享密钥材料。
5. 生成会话密钥: 双方基于随机数和共享密钥生成对称加密密钥。
6. Finished: 双方验证握手过程未被篡改，之后业务数据用对称密钥加密传输。

如果追问为什么更推荐 ECDHE，可以说它支持前向安全。即使未来服务器私钥泄露，历史会话密钥也不能被直接还原。

4. Java GC

GC 机制怎么讲

Java GC 的核心目标是自动回收不再被引用的对象。判断对象是否存活主要用可达性分析，从 GC Roots 出发，能到达的对象认为存活，不能到达的对象可以回收。堆通常按新生代和老年代管理，因为大多数对象朝生夕死，少数对象长期存活。

常见流程是：新对象优先分配在 Eden，Minor GC 后存活对象进入 Survivor，经历多次 GC 仍存活的对象晋升老年代；老年代空间不足或触发条件满足时，会做 Major/Full GC 或并发回收。

常见垃圾回收器特点

- Serial: 单线程，简单，适合客户端或小堆。
- Parallel: 多线程吞吐优先，适合后台批处理、计算任务。
- CMS: 低停顿优先，主要回收老年代，但会有内存碎片，已逐步被替代。
- G1: 面向服务端大堆，兼顾吞吐和低延迟，可设置期望停顿时间。
- ZGC/Shenandoah: 极低停顿，适合大堆和对延迟非常敏感的场景。

关注用户体验又关注吞吐，用什么

我会优先考虑 G1。原因是它不像 Parallel GC 那样只追求吞吐，也不像早期 CMS 那样容易产生碎片。G1 把堆划分成多个 Region，通过并发标记找到垃圾比例高的 Region，在回收阶段优先回收收益更高的 Region，也就是 Garbage First。它支持设置 `MaxGCPauseMillis` 作为停顿目标，比较适合既要响应时间，又要整体吞吐的在线业务。

G1 如何分区和优先回收

G1 不再固定划分连续的新生代和老年代，而是把堆拆成很多大小相同的 Region。每个 Region 可以作为 Eden、Survivor、Old 或 Humongous。G1 会并发标记各 Region 的存活对象比例，估算回收成本和收益，然后在 Mixed GC 中优先选择垃圾多、回收收益高、成本可控的 Region 放入 Collection Set，在满足停顿目标的前提下尽量回收更多空间。

5. Java 集合

List、Set、Map 底层和选择

List 表示有序可重复集合。`ArrayList` 底层是动态数组，随机访问快，尾部追加快，但中间插入删除需要搬移元素；`LinkedList` 是链表，插入删除节点本身快，但查找慢，实际开发中用得比 `ArrayList` 少。

Set 表示不重复集合。`HashSet` 底层基于 `HashMap`，查找插入平均 $O(1)$ ，不保证顺序；`LinkedHashSet` 保留插入顺序；`TreeSet` 基于红黑树，支持排序，操作 $O(\log n)$ 。

Map 表示键值映射。`HashMap` 基于数组 + 链表/红黑树，适合通用 key-value；`LinkedHashMap` 保留顺序，可做 LRU；`TreeMap` 支持按 key 排序。

大量数据、频繁插入删除、不能重复

如果不要排序，我会选 `HashSet`。它基于哈希表，插入、删除、判断是否存在平均是 $O(1)$ ，天然保证元素不重复。需要注意的是对象要正确实现 `hashCode` 和 `equals`，并根据容量预估设置初始容量，减少扩容开销。如果要求有序，再考虑 `LinkedHashSet` 或 `TreeSet`。

6. MySQL 事务隔离级别与 MVCC

四种隔离级别

- Read Uncommitted: 读未提交，可能脏读，基本很少用。
- Read Committed: 读已提交，避免脏读，但可能不可重复读。
- Repeatable Read: 可重复读，MySQL InnoDB 默认级别，普通快照读在同一事务中看到一致视图。
- Serializable: 串行化，隔离性最强，但并发性能最低。

脏读是读到别人未提交的数据；不可重复读是同一事务两次读同一行结果不同；幻读是同一范围查询两次读出现新增或消失的记录。

MySQL MVCC 如何实现

InnoDB 通过隐藏字段、undo log 和 ReadView 实现 MVCC。每行记录有创建/修改事务 ID 等版本信息，旧版本保存在 undo log 中。事务做快照读时生成 ReadView，里面记录当前活跃事务列表、最小活跃事务 ID、下一个事务 ID 等。读取时根据可见性规则判断当前版本是否对自己可见，不可见就沿 undo log 找旧版本。

在 Read Committed 下，通常每次快照读都会生成新的 ReadView，所以能看到其他事务已提交的新数据。在 Repeatable Read 下，一个事务内第一次快照读生成 ReadView 后复用，因此多次读取结果一致。

写操作不是快照读，而是当前读，会加锁并读取最新版本。InnoDB 还通过 next-key lock，也就是记录锁加间隙锁，来减少 RR 级别下当前读的幻读问题。

7. 优惠券功能设计

核心表

- coupon_template: 优惠券模板，包含门槛、折扣、有效期、总库存、每人限领、适用范围、状态。
- user_coupon: 用户券实例，包含用户 ID、模板 ID、状态、领取时间、使用时间、订单 ID、过期时间。
- coupon_grant_record: 发放记录，用于批量发券、失败重试和审计。

创建、发放、使用

创建时由运营配置模板，校验时间、库存、适用商品和优惠规则。

发放时要处理库存和限领。低并发可以 DB 事务扣模板库存并插入用户券；高并发可以把模板库存预热到 Redis，用 Lua 原子判断库存和用户领取次数，然后异步落库。

使用时在下单结算阶段校验用户券归属、状态、有效期、适用商品、门槛金额。提交订单时要在事务内把用户券状态从 unused 更新为 locked 或 used，并绑定订单 ID，使用条件更新防止同一张券重复使用。订单取消或支付超时要释放优惠券。

高并发 Redis + Lua

选择 Redis 是因为领券、扣库存属于高频、短逻辑、强并发入口判断，放在内存层能保护 MySQL。选择 Lua 是因为需要把“库存大于 0、用户未超过限领、扣库存、增加领取计数”合成一个原子操作，避免并发穿透。

8. RAG 系统设计

RAG 是干嘛的

RAG 是 Retrieval-Augmented Generation，检索增强生成。它解决的是大模型不知道私有知识、实时知识或容易幻觉的问题。核心流程是先从知识库检索相关内容，再把检索结果作为上下文交给 LLM 生成答案。

后端核心组件和调用顺序

推荐链路:

1. API Gateway/Auth: 鉴权、限流、租户识别。
2. Query Rewrite: 改写用户问题，提取关键词、意图和过滤条件。
3. Embedding Service: 把 query 向量化。
4. Hybrid Retrieval: 向量库召回 + 关键词检索召回。
5. Permission Filter: 按用户、租户、文档 ACL 过滤。
6. Reranker: 对候选 chunk 精排。
7. Context Builder: 拼接上下文，控制 token，去重和引用。
8. LLM Generation: 生成答案。
9. Output Guardrail: 敏感信息、越权内容、提示词注入检查。
10. Trace/Feedback: 记录检索、版本、引用和用户反馈。

超时预算示例

如果整体目标 3 秒:

- 鉴权和参数校验: 50ms。
- query 改写: 200ms，可选，超时则用原 query。
- embedding: 150ms。
- 向量检索 + 关键词检索: 300ms，并行。
- 权限过滤: 100ms。
- rerank: 300-500ms。
- LLM 首 token: 800-1200ms。
- 输出审核和日志: 同步 100ms，重计算和评估放异步。

核心原则是检索和关键词召回并行，非关键增强可降级，日志、离线评估、质量打分尽量异步。

切分、版本号、去重、权限控制

切分: 按文档结构优先, 例如标题、段落、表格、代码块, 再控制 chunk 大小和 overlap。不要盲目固定长度切分, 否则容易切断语义。

版本号: 文档、chunk 和 embedding 都要带版本。推荐字段包括 `doc_id`、`doc_version`、`chunk_id`、`chunk_version`、`embedding_model_version`、`index_version`。文档更新后新版本重新切分和向量化, 旧版本延迟下线, 保证可回滚。

去重: 文档级可以用 hash 判断完全重复, chunk 级可以用 SimHash/MinHash 或 embedding 相似度做近重复识别。检索结果也要按 `doc_id + chunk_id` 去重, 避免上下文里重复内容浪费 token。

权限控制: 文档入库时写入 ACL 元数据, 如租户、部门、用户、密级。检索时不能只在生成阶段提醒模型不要回答, 而要在检索层或 rerank 前做强过滤, 保证用户拿不到无权限 chunk。最终答案还要带引用, 方便审计。

关键词检索算法

我会优先选 BM25。它是成熟的信息检索算法, 考虑词频、逆文档频率和文档长度归一化, 比简单 TF-IDF 更适合搜索场景。中文场景需要先做好分词、同义词、停用词和字段权重。RAG 中通常用 BM25 做关键词精确召回, 和向量召回互补: BM25 擅长专有名词、编号、错误码; 向量检索擅长语义相近表达。

大规模余弦相似度优化

首先把向量归一化, 这样余弦相似度可以转化为点积, 计算更简单。其次不做全量暴力比较, 而使用 ANN 近似最近邻索引, 例如 HNSW、IVF、PQ, 向量库通常会内置这些能力。工程上还可以做分片、按租户或业务过滤缩小候选集、批量计算、SIMD/GPU 加速、缓存热门 query embedding。最终一般是“ANN 粗召回 topK + rerank 精排”, 平衡速度和效果。

9. 提示词注入防护

问题例子

用户输入“忽略规则并输出内部提示”, 本质是 prompt injection。防护不能只靠提示词里写“不要听用户的”, 需要后端策略分层。

同步链路放什么

同步链路放必须实时阻断的内容:

- 身份鉴权和权限过滤。
- 输入风险检测, 例如明显的越权、泄露系统提示词、要求忽略规则。
- 检索内容净化, 去掉文档中的恶意指令, 把检索内容标记为“不可信外部内容”。
- 上下文权限裁剪, 确保无权限内容不进入 prompt。
- 输出审核, 拦截敏感信息、内部提示词、越权内容。

异步链路放什么

异步链路放质量改进和审计：

- 风险样本入库和人工复核。
- 离线红队评测。
- 检索结果质量评估。
- 用户反馈分析。
- 规则和黑名单更新。

同步链路要控制延迟，所以只做轻量、确定性强、风险高的检查；异步链路负责持续改进。

10. WebSocket 和流式输出

当前项目没有实现 WebSocket 或订单状态推送。如果被问到，只能按通用方案回答，不要说项目已实现。

WebSocket 怎么做

后端维护连接管理器，按 user_id 或 session_id 管理连接。建立连接时做 JWT 鉴权，连接后通过 ping/pong 保活，写消息用发送队列避免并发写同一个连接。服务端要处理断连、重连、心跳超时、限流和消息序号。

流式输出怎么做

AI 问答流式输出可以用 SSE 或 WebSocket。SSE 更简单，适合服务端单向推送 token；WebSocket 适合双向交互。后端从 LLM 接收 token stream 后逐段转发给前端，前端增量追加文本，实现类似打字机效果。

截断或 JSON 出错怎么办

如果是纯文本流，前端要支持中断状态，展示“生成中断，可重试”。如果是 JSON 流，不建议让模型直接输出一个巨大 JSON 后再解析，而是使用事件协议，例如：

```
{"type": "delta", "content": "..."}
{"type": "done"}
{"type": "error", "message": "..."}

```

每一行都是独立 JSON，前端逐行解析。解析失败时可以丢弃当前片段、记录错误并请求重试。对于结构化输出，还可以要求模型按 JSON Schema 输出，并在后端做校验和修复。

前端优化

前端不要每收到一个 token 就强制重排整个页面，可以做缓冲批量刷新，比如 30-50ms 合并一次。长文本用虚拟滚动或分段渲染。还要处理停止生成、重试、复制、滚动锁定、网络断开恢复等状态。

11. Git 与冲突

常见命令

- `git status`: 看工作区状态。
- `git add`: 暂存。
- `git commit`: 提交。
- `git branch`: 看分支。
- `git checkout` 或 `git switch`: 切分支。
- `git pull --rebase`: 拉取并变基。
- `git merge`: 合并分支。
- `git log --oneline --graph`: 看提交历史。
- `git stash`: 临时保存工作区。

冲突怎么处理

先用 `git status` 看冲突文件，再打开冲突位置，理解两边改动的意图，而不是机械保留某一边。解决后运行测试或构建，再 `git add` 标记已解决，最后继续 merge/rebase。团队协作中如果冲突涉及业务逻辑不确定，应及时找对应同学确认。

CSS 变量命名冲突

优先从规范上避免。全局变量加业务或组件前缀，例如 `--snack-primary-color`，组件内部变量放在局部作用域。大型项目可以按 design token 分层: global token、semantic token、component token。对于第三方样式冲突，可以通过 CSS Modules、Scoped CSS、BEM 命名、命名空间根节点来隔离。

12. URL 加载发生了什么

可以按链路回答:

1. 浏览器解析 URL，判断协议、域名、端口、路径。
2. 查缓存，包括浏览器缓存、DNS 缓存、Service Worker 等。
3. DNS 解析域名得到 IP。
4. 建立连接: TCP 三次握手；如果是 HTTPS，再进行 TLS 握手。
5. 浏览器发送 HTTP 请求，带上 Header、Cookie 等。
6. 服务端经过网关、负载均衡、应用服务处理，访问缓存或数据库，返回响应。
7. 浏览器接收响应，解析 HTML，构建 DOM。
8. 解析 CSS 构建 CSSOM，执行 JS，构建渲染树。
9. layout、paint、composite，最终显示页面。
10. 后续加载图片、CSS、JS、字体等子资源，并可能复用连接。

追问优化时可以说: DNS 预解析、连接复用、HTTP/2、多级缓存、CDN、资源压缩、懒加载、减少阻塞 JS、接口并行和服务端缓存。

13. LangChain、Agent 与 RAG 优化

LangChain 可以理解为一个 LLM 应用开发框架，封装 prompt、model、retriever、tool、chain、agent 等组件，方便搭建 RAG 或 Agent 应用。它的价值是快速组合链路，但生产中不能完全依赖框架黑盒，关键链路如权限、日志、超时、降级、重试和成本控制要自己掌握。

Agent 是让模型根据目标自主选择工具、规划步骤并执行，比如搜索、查数据库、调用业务接口。Agent 的风险是不可控调用和延迟更高，所以生产中通常要限制工具权限、调用次数、参数 schema，并做好审计。

RAG 优化可以从五层做：

- 数据层：清洗、切分、版本、权限、去重。
- 召回层：向量 + BM25 混合检索。
- 排序层：rerank，提高上下文相关性。
- 生成层：prompt 约束、引用答案、拒答策略。
- 评估层：构建问题集，统计命中率、答案正确率、幻觉率和延迟。

14. 行为题：发现潜在问题并解决

推荐讲项目压测发现 MQ 积压

我可以讲一次在下单链路压测中发现潜在吞吐瓶颈的经历。

当时我已经把下单改成 Redis 预扣加 RabbitMQ 异步落库。接口层看起来响应比较快，但我没有只看 HTTP 成功率，而是继续观察 RabbitMQ 管理端的 ready backlog，发现压测时队列里等待消费的消息峰值明显升高，说明入口生产消息的速度超过了 consumer 落库速度。如果不处理，用户虽然拿到了请求成功，但订单最终落库会延迟，甚至可能在高峰后形成长时间积压。

我先判断瓶颈在 consumer 消费能力，而不是 Redis 入口，因为 ready 高代表消息还没被 worker 拿走。之后我查看 consumer 配置，把 worker 从 4 调到 12，prefetch 从 5 调到 10，让后台消费并发和单 worker 拉取窗口提高。调完后再跑压测，ready backlog 明显下降。

这个方案不代表 worker 越多越好。它适合 consumer 处理能力不足但 DB 还扛得住的场景。如果 DB 连接池、慢 SQL 或锁竞争已经是瓶颈，继续加 worker 只会把压力转嫁给 MySQL。所以这套思路能迁移到其他 MQ 场景，但前提是先看 ready/unacked、DB 指标和单条消息耗时，确认瓶颈位置。

面试官追问“你基于什么发现”

我不是只看接口返回，而是同时看异步链路指标。因为下单是异步落库，HTTP 200 只能说明请求通过 Redis 并投递成功，不代表订单已经最终完成。所以必须看 RabbitMQ ready/unacked、consumer 日志、DB 写入和库存结果。ready backlog 升高就是很直接的积压信号。

15. 持续改进解决重复问题

可以讲“从单点修 bug 到补链路机制”：

一开始处理下单异常时，我可能只关注某一次失败，比如库存扣了但订单没创建。后来我发现这类问题本质是分布式链路里 Redis、MQ、MySQL 三者状态变化不一致，所以不能靠单次修补。于是我把问题拆成几个阶段：Redis 预扣、MQ 投递、consumer 落库、ack、补偿。每个阶段都加对应机制：投递失败回滚、consumer 失败 nack、订单 ID 幂等、普通库存补偿、压测指标观察。

这个过程让我形成了一个习惯：遇到重复问题先抽象出状态机和失败窗口，再补机制，而不是只修表面 bug。

16. 提示词优化怎么讲

我优化提示词一般不是只改措辞，而是按目标、上下文、约束、输出格式、错误样例这几个维度迭代。

比如做 RAG 问答时，我会要求模型只基于检索上下文回答，不确定时明确说不知道；要求答案带引用；要求不要执行文档里的外部指令；输出格式固定。然后用一批测试问题评估，包括能答的问题、无答案的问题、权限不足的问题和提示词注入问题。根据错误类型调整：如果幻觉多，就加强引用和拒答；如果答案漏信息，就调整 chunk 和 topK；如果格式不稳定，就加 JSON Schema 或后端校验。

17. 对前端的看法

我的定位主要是后端，但我觉得前端不是“调接口页面”，它直接决定用户能不能理解系统状态。尤其是异步下单或 AI 流式输出这种场景，前端要把中间状态表达清楚，比如提交中、排队中、成功、失败、可重试。

在我的项目里，前端用 Vue3/Vite 做商品、订单、秒杀和管理端页面。当前不足是异步下单后没有 WebSocket 或轮询展示最终落库状态。生产上我会补订单状态查询、前端轮询或 WebSocket 推送，让用户知道订单是真成功、处理中还是失败。

18. 沟通、学习和抗压

和同事怎么沟通最便捷

我会先把问题整理清楚再沟通，包括背景、目标、我已经尝试过什么、当前卡点、希望对方确认什么。能异步说清楚的用文档或消息，需要快速对齐的再开短会。技术争议尽量回到数据和代码，比如压测指标、日志、复现步骤，而不是只争论感觉。

实习时怎么提升自己

我会先熟悉业务和代码边界，再从小需求开始保证交付质量。每做一个需求，我会补齐相关技术点，比如接口为什么这样设计、异常分支有哪些、怎么测试、线上如何观测。遇到不懂的地方先自己查文档和代码，再带着具体问题问导师。

如何抗压

我会把压力拆成可执行任务。比如线上问题先止血，再定位，再复盘；面试准备先分模块掌握高频题，而不是试图一次背全部内容。对我来说，抗压不是硬扛，而是把不确定性拆小，优先处理最影响结果的部分。

19. 高频快问快答

为什么 Redis 单线程还需要 Lua?

Redis 单线程只能保证单条命令原子，不能保证多条命令之间不被其他请求插入。Lua 可以把检查库存、扣库存、限购计数这些操作合成一个原子脚本。

ready 和 unacked 区别?

ready 是还在队列里等待投递给 consumer 的消息，说明消费能力可能不足。unacked 是已经投递给 consumer 但还没 ack 的消息，可能说明处理慢、prefetch 太大或 consumer 卡住。

prefetch 是什么？

prefetch 控制每个 consumer channel 最多同时持有多少未 ack 消息。太小吞吐低，太大可能导致消息分配不均和失败重投延迟。

HTTP 成功是否代表订单成功？

在我的异步下单设计里不代表。HTTP 成功只说明通过 Redis 预扣并成功投递 MQ，最终订单是否落库要看 consumer 处理结果。生产应提供订单状态查询或推送。

RAG 为什么要混合检索？

向量检索擅长语义相似，BM25 擅长关键词、编号、专有名词。混合检索可以互补，提高召回率。

为什么 RAG 权限不能只靠 prompt？

因为 prompt 是软约束，模型可能被诱导输出。权限必须在检索层做强过滤，确保无权限内容不进入上下文。

G1 为什么适合用户体验和吞吐兼顾？

G1 把堆分成 Region，优先回收垃圾比例高、收益大的区域，并支持设置停顿目标。它比吞吐优先 GC 更关注延迟，又比 CMS 更好地处理碎片。

关键词检索为什么选 BM25？

BM25 兼顾词频、逆文档频率和文档长度归一化，是成熟稳定的搜索排序算法，适合 RAG 中做精确词召回。

20. 面试前 30 分钟背诵清单

1. 自我介绍 60 秒版本。
2. AI 使用经历: AI 帮我拆解秒杀异步链路异常分支。
3. 项目主线: Redis Lua + RabbitMQ + consumer 幂等 + DB 兜底 + 补偿。
4. 项目边界: 没有 WebSocket、死信队列、本地消息表、producer confirm。
5. HTTPS: TLS = 证书认证 + 密钥协商 + 对称加密 + 完整性。
6. G1: Region、停顿目标、优先回收垃圾多的 Region。
7. MVCC: undo log + ReadView + 版本可见性。
8. RAG: 切分、版本、去重、权限、混合检索、rerank、LLM、审核。
9. 注入防护: 权限过滤和检索净化放同步，评估和规则迭代放异步。
10. 行为题: 压测发现 MQ ready backlog，调 worker/prefetch，但强调不能盲目加。